4.2 Classification Models

This section explains the implementation of classification models used in the FURSAN framework to categorise satellite conjunction events into risk levels. While regression models predict continuous collision probability values, classification models are designed to classify events into discrete categories such as low-risk and high-risk collisions. This enables faster decision-making and supports operational satellite safety systems.

4.2.1 Overview

Classification models were developed to categorise satellite conjunction events into discrete collision risk levels. This approach enables the FURSAN framework to classify events into low-risk, medium-risk, and high-risk categories using orbital parameters, covariance data, and derived physics-based features. Classifying risk levels allows satellite operators to quickly identify critical conjunction events and prioritise collision avoidance actions.

4.2.2 Model Development

Multiple classification algorithms were implemented to evaluate performance and robustness. These models were trained using the engineered features generated during the preprocessing stage. The dataset was split into training and testing sets, and feature scaling was applied where required before model training.

Classification-based risk categorisation was performed by converting continuous collision probability values into discrete classes using defined thresholds. This approach enables the models to learn patterns associated with different collision risk levels and improves decision-making for high-risk satellite conjunction events.

Models used:

Model	Type	Purpose
Logistic Regression	Machine Learning	Baseline binary classification model
Random Forest Classifier	Machine Learning	Ensemble tree classifier for risk categorisation
XGBoost Classifier	Machine Learning	High-performance gradient boosting classifier
LightGBM Classifier	Machine Learning	Efficient classifier for large tabular datasets
CatBoost Classifier	Machine Learning	Handles categorical features and feature interactions
Feedforward Neural Network (FNN) Classifier	Deep Learning	Basic neural network for risk classification
Multilayer Perceptron (MLP) Classifier	Deep Learning	Deep learning baseline for tabular classification
Deep Neural Network (DNN) Classifier	Deep Learning	Captures complex nonlinear risk patterns
Transformer-Based Classifier (TabTransformer)	Deep Learning	Attention-based classification for tabular data
Autoencoder + Classifier	Deep Learning	Detects anomalies and classifies high-risk events
LSTM Classifier	Deep Learning	Classify temporal sequences of CDM updates

Table 1: Classification Models Evaluated for High-Risk and Low-Risk Conjunction Event Categorisation

4.2.3 Evaluation matrix, criteria used and final results:

A second set of models was then tested to predict high- vs. low-risk events using thresholder regression labels. Evaluation uses Accuracy, F1-macro, Precision, Recall, and runtime

Model	Accuracy	Precision	Recall	F1	F2	Runtime (s)
LightGBM Classifier	0.997181833	0.997170832	0.997181833	0.99717086	0.997176165	1644.368617
XGBoost Classifier	0.996528345	0.996526621	0.996528345	0.99652723	0.996527838	1622.943331
CatBoost Classifier	0.99632413	0.996320618	0.99632413	0.996321484	0.99632286	1564.052663
Random Forest Classifier	0.975126613	0.974877596	0.975126613	0.972121597	0.973754342	822.8912725
MLP Classifier	0.928769809	0.925006729	0.928769809	0.926345753	0.927682205	1755.493341
Deep Neural Network Classifier	0.926523444	0.925098067	0.926523444	0.925434711	0.92602839	292.8000312
Feedforward NN Classifier	0.927340304	0.923825657	0.927340304	0.925029884	0.926305691	212.3842504
Logistic Regression (Baseline)	0.902630289	0.889293403	0.902630289	0.890154115	0.896683221	1455.158076
Autoencoder + Classifier	0.900588139	0.886664622	0.900588139	0.886876761	0.894040806	399.3545909
LSTM Classifier	0.881636987	0.850068916	0.881636987	0.848284947	0.866459284	587.6058884
TabTransformer Classifier	0.866770136	0.751290468	0.866770136	0.804909457	0.840918874	2085.511072

Table 2: Classification Model Performance for High-Risk vs Low-Risk Collision Events

Models were compared using:

1. Highest Accuracy
2. Highest Precision
3. Highest Recall (ability to detect high-risk events)
4. Best F1 / F2 scores
5. Reasonable runtime

LightGBM achieves the best overall classification performance, producing the highest accuracy, precision, recall, F1, and F2 scores among all evaluated models while maintaining moderate runtime. Although XGBoost and CatBoost perform similarly, LightGBM provides slightly better predictive consistency across all evaluation metrics, making it the most reliable model for distinguishing high-risk from low-risk collision events.

Tree-based ensemble classifiers generally outperform other approaches, indicating that gradient boosting methods are highly effective for tabular conjunction data with complex nonlinear feature interactions. The Random Forest classifier performs reasonably well but shows slightly lower accuracy and F1 scores compared to the boosting models.

Deep learning classifiers such as Feedforward Neural Networks, Multilayer Perceptrons, and Deep Neural Networks achieve moderate performance but do not surpass gradient boosting models despite increased computational complexity. More advanced architectures, such as the TabTransformer and LSTM classifier, exhibit lower predictive accuracy and significantly higher runtime, suggesting that attention mechanisms and sequential modelling do not substantially improve classification performance for this dataset, which is primarily structured tabular data rather than sequential time-series data.

Logistic Regression performs adequately as a baseline model but produces noticeably lower accuracy and F1 scores, confirming that collision risk classification involves nonlinear relationships that are not well captured by simple linear decision boundaries. Overall, the results demonstrate that gradient boosting classifiers, particularly LightGBM, provide the most effective and computationally efficient solution for high-risk versus low-risk event classification

4.2.4 Code used:

Code Section 1: Library Imports and Environment Setup

The classification model implementation begins with importing the required libraries. These libraries include data processing tools, machine learning models, deep learning frameworks, and evaluation metrics.

NumPy and Pandas are used for data manipulation and numerical operations, while Scikit-learn provides preprocessing tools, classification algorithms, and performance evaluation metrics. Additional machine learning models such as XGBoost, LightGBM, and CatBoost are imported to improve classification performance. TensorFlow and Keras are used to implement deep learning models including neural networks, LSTM models, and transformer-based architectures.

Parallel processing libraries are also imported to allow multiple models to run simultaneously, reducing overall computation time. CPU thread control is applied to prevent oversubscription and improve system stability during model training.

```
import numpy as np
import pandas as pd
import time
import os

from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, fbeta_score
)

# Machine Learning Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# Deep Learning
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Input, LSTM, Flatten, MultiHeadAttention, LayerNormalization
from tensorflow.keras.utils import to_categorical

os.environ["OMP_NUM_THREADS"] = "4"
```

Figure 4.2.4.1.1: Import Libraries and Environment Setup

Code Section 2: Risk Class Generation

The next step involves converting continuous collision probability values into classification categories. A custom function is implemented to assign three risk levels:

- Low Risk (0)
- Medium Risk (1)
- High Risk (2)

The classification thresholds are defined based on logarithmic collision probability values. These thresholds divide the dataset into meaningful risk categories, allowing classification models to learn patterns associated with dangerous conjunction events.

After generating risk classes, the distribution of training and testing labels is printed. This helps verify class balance and ensures that models are trained effectively.

```python
# ====================================================
# CREATE RISK CLASSES
# ====================================================

def create_risk_class(y_log):

    """
    0 = Low Risk
    1 = Medium Risk
    2 = High Risk
    """

    conditions = [
        y_log > -4,
        (y_log <= -4) & (y_log > -6),
        y_log <= -6
    ]

    return np.select(conditions, [0,1,2]).astype(int)

y_train = create_risk_class(train_data["risk"])
y_test = create_risk_class(test_data["risk"])

print("Train distribution:", np.bincount(y_train))
print("Test distribution:", np.bincount(y_test))
```

Figure 4.2.4.2.1: Risk Class Creation Function

Code Section 3: Feature Matrix Preparation

After generating classification labels, the dataset is separated into input features and target variables. The risk column is removed from the feature matrix, leaving only input features used for model training.

Categorical features are converted into numerical values using one-hot encoding. This transformation ensures compatibility with machine learning models. The training and testing datasets are then aligned to ensure both datasets contain identical features.

Missing values and infinite values are handled using median replacement. Feature values are clipped to prevent extreme values from affecting model performance. Finally, all features are converted to numerical format for compatibility with classification models.

```python
# ====================================================
# FEATURE MATRIX
# ====================================================

X_train = train_data.drop(columns=["risk"])
X_test = test_data.drop(columns=["risk"])

# One-hot encode categorical features
X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)

X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)

# Handle inf / NaN
X_train = X_train.replace([np.inf, -np.inf], np.nan)
X_test = X_test.replace([np.inf, -np.inf], np.nan)

median_vals = X_train.median()

X_train = X_train.fillna(median_vals)
X_test = X_test.fillna(median_vals)

X_train = X_train.clip(-1e6,1e6)
X_test = X_test.clip(-1e6,1e6)

X_train = X_train.astype(np.float32)
X_test = X_test.astype(np.float32)
```

Figure 4.2.4.3.1: Feature Matrix Preparation Code

Code Section 4: Feature Scaling

Feature scaling is applied using standardisation to normalise the dataset. Standard scaling improves model performance by ensuring all features have consistent value ranges. This is particularly important for neural networks and linear classification models.

The scaler is fitted using training data and then applied to testing data to avoid data leakage. In addition, classification targets are converted into one-hot encoded format for deep learning models.

```
# ====================================================
# FEATURE SCALING
# ====================================================

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled = np.nan_to_num(X_train_scaled)
X_test_scaled = np.nan_to_num(X_test_scaled)

# One-hot targets for deep learning
y_train_dl = to_categorical(y_train, 3)
y_test_dl = to_categorical(y_test, 3)
```

Figure 4.2.4.4.1: Feature Scaling Code

Code Section 5: Classification Results Storage

A results list is created to store classification model performance metrics. This structure allows performance comparison between different classification models. Each model's evaluation metrics and runtime are stored in this list.

```
# ====================================================
# RESULTS STORAGE
# ====================================================

classification_results = []
```

Figure 4.2.4.5.1: Results Storage Code

Code Section 6: Classification Evaluation Function

An evaluation function is implemented to measure classification performance. This function calculates key metrics including:

- Accuracy
- Precision
- Recall
- F1 Score
- F2 Score

Accuracy measures overall correctness, while precision measures prediction reliability. Recall evaluates the model's ability to detect high-risk events. The F1 score balances precision and recall, while F2 score prioritises recall, which is important for safety-critical satellite collision prediction.

The results are stored in the classification results list for later comparison.

```python
# ====================================================
# EVALUATION FUNCTION
# ====================================================

def evaluate_classifier(name, y_true, y_pred, runtime):

    acc = accuracy_score(y_true, y_pred)

    precision = precision_score(y_true, y_pred, average="weighted")
    recall = recall_score(y_true, y_pred, average="weighted")

    f1 = f1_score(y_true, y_pred, average="weighted")

    f2 = fbeta_score(y_true, y_pred, beta=2, average="weighted")

    classification_results.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": precision,
        "Recall": recall,
        "F1": f1,
        "F2": f2,
        "Runtime (s)": runtime
    })

    print(f"{name} completed in {runtime:.2f} seconds")
```

Figure 4.2.4.6.1: Evaluation Function Code

Code Section 7: Machine Learning Classification Models

Several machine learning classification models are implemented to evaluate performance across different algorithms. These models include baseline models, ensemble methods, and neural network classifiers.

The classification models implemented include:

- Logistic Regression
- Random Forest Classifier
- XGBoost Classifier
- LightGBM Classifier
- CatBoost Classifier
- MLP Classifier

Each model is trained using the training dataset and evaluated using the defined metrics. Parallel processing is used to execute models simultaneously, reducing training time.

```python
def run_ml_classifier(name, model, Xtr, Xte, ytr, yte):

    start = time.time()

    model.fit(Xtr, ytr)

    pred = model.predict(Xte)

    runtime = time.time() - start

    evaluate_classifier(name, yte, pred, runtime)

ml_models = [

    # Logistic Regression
    ("Logistic Regression (Baseline)",
     LogisticRegression(max_iter=1000),
     X_train_scaled, X_test_scaled),

    # Random Forest
    ("Random Forest Classifier",
     RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1),
     X_train, X_test),

    # XGBoost
    ("XGBoost Classifier",
     XGBClassifier(n_estimators=500, learning_rate=0.05, max_depth=6),
     X_train, X_test),

    # LightGBM
    ("LightGBM Classifier",
     LGBMClassifier(n_estimators=500, learning_rate=0.05),
     X_train, X_test),
```

```python
    # CatBoost
    ("CatBoost Classifier",
     CatBoostClassifier(iterations=500, depth=6, verbose=0),
     X_train, X_test),

    # MLP Classifier
    ("MLP Classifier",
     MLPClassifier(hidden_layer_sizes=(128,64), max_iter=500),
     X_train_scaled, X_test_scaled)
]

print("\nRunning Machine Learning Classification Models...\n")

with ThreadPoolExecutor(max_workers=6) as executor:

    futures = []

    for name, model, Xtr, Xte in ml_models:
        futures.append(
            executor.submit(run_ml_classifier, name, model, Xtr, Xte, y_train, y_test)
        )

    for future in as_completed(futures):
        future.result()
```

Figure 4.2.4.7.1: Machine Learning Classification Models Code

Code Section 8: Deep Learning Classification Models

In addition to traditional machine learning models, deep learning models are implemented to capture complex relationships in satellite conjunction data. These models include multiple neural network architectures.

The deep learning classification models implemented include:

- Feedforward Neural Network
- Deep Neural Network
- LSTM Classifier
- TabTransformer Model
- Autoencoder with Logistic Regression Classifier

Each model is trained using the scaled dataset and one-hot encoded targets. Predictions are generated and converted into class labels before evaluation.

```python
# ====================================================
# DEEP LEARNING MODELS
# ====================================================

print("\nRunning Deep Learning Classification Models...\n")

# ----------------------------------------------------
# Feedforward Neural Network
# ----------------------------------------------------

def run_fnn():

    start = time.time()

    model = Sequential([
        Input(shape=(X_train_scaled.shape[1],)),
        Dense(64, activation='relu'),
        Dense(32, activation='relu'),
        Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    model.fit(X_train_scaled, y_train_dl, epochs=25, batch_size=32, verbose=0)

    pred = model.predict(X_test_scaled)

    pred_classes = np.argmax(pred, axis=1)

    return ("Feedforward NN Classifier", pred_classes, time.time()-start)
```

```python
# ----------------------------------------------------
# Deep Neural Network
# ----------------------------------------------------

def run_dnn():

    start = time.time()

    model = Sequential([
        Input(shape=(X_train_scaled.shape[1],)),
        Dense(128, activation='relu'),
        Dense(128, activation='relu'),
        Dense(64, activation='relu'),
        Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    model.fit(X_train_scaled, y_train_dl, epochs=30, batch_size=32, verbose=0)

    pred = model.predict(X_test_scaled)

    pred_classes = np.argmax(pred, axis=1)

    return ("Deep Neural Network Classifier", pred_classes, time.time()-start)
```

```python
# ----------------------------------------------------
# LSTM CLASSIFIER
# ----------------------------------------------------

def run_lstm():

    start = time.time()

    Xtr = X_train_scaled.reshape(-1, X_train_scaled.shape[1],1)
    Xte = X_test_scaled.reshape(-1, X_test_scaled.shape[1],1)

    model = Sequential([
        LSTM(32),
        Dense(16, activation='relu'),
        Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    model.fit(Xtr, y_train_dl, epochs=10, batch_size=64, verbose=0)

    pred = model.predict(Xte)

    pred_classes = np.argmax(pred, axis=1)

    return ("LSTM Classifier", pred_classes, time.time()-start)
```

```python
# TABTRANSFORMER
# ----------------------------------------------------
def run_tabtransformer():
    start = time.time()
    inputs = Input(shape=(X_train_scaled.shape[1],1))
    attention = MultiHeadAttention(num_heads=4, key_dim=16)(inputs, inputs)
    norm = LayerNormalization()(attention)
    flat = Flatten()(norm)
    dense = Dense(64, activation="relu")(flat)
    output = Dense(3, activation="softmax")(dense)

    model = Model(inputs, output)

    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    model.fit(
        X_train_scaled.reshape(-1,X_train_scaled.shape[1],1),
        y_train_dl,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    pred = model.predict(
        X_test_scaled.reshape(-1,X_test_scaled.shape[1],1)
    )

    pred_classes = np.argmax(pred, axis=1)

    return ("TabTransformer Classifier", pred_classes, time.time()-start)
```

```python
# ----------------------------------------------------
# AUTOENCODER + CLASSIFIER
# ----------------------------------------------------

def run_autoencoder():

    start = time.time()

    input_layer = Input(shape=(X_train_scaled.shape[1],))

    encoded = Dense(128, activation="relu")(input_layer)
    encoded = Dense(64, activation="relu")(encoded)

    decoded = Dense(128, activation="relu")(encoded)
    decoded = Dense(X_train_scaled.shape[1])(decoded)

    autoencoder = Model(input_layer, decoded)

    autoencoder.compile(optimizer="adam", loss="mse")

    autoencoder.fit(X_train_scaled, X_train_scaled, epochs=20, batch_size=32, verbose=0)

    encoder = Model(input_layer, encoded)

    X_train_latent = encoder.predict(X_train_scaled)
    X_test_latent = encoder.predict(X_test_scaled)

    clf = LogisticRegression(max_iter=1000)

    clf.fit(X_train_latent, y_train)

    pred = clf.predict(X_test_latent)

    return ("Autoencoder + Classifier", pred, time.time()-start)
```

Figure 4.2.4.8.1: Deep Learning Models Code

Code Section 9: Model Execution

All deep learning models are executed sequentially. Runtime is recorded for each model to compare computational efficiency. The predictions from each model are evaluated using the classification evaluation function.

```python
# ====================================================
# RUN DEEP MODELS
# ====================================================

dl_models = [
    run_fnn,
    run_dnn,
    run_lstm,
    run_tabtransformer,
    run_autoencoder
]

for model in dl_models:

    name, pred, runtime = model()

    evaluate_classifier(name, y_test, pred, runtime)
```

Figure 4.2.4.9.1: Model Execution Code

Code Section 10: Model Performance Comparison

After training all models, results are stored in a structured dataframe. The dataframe is sorted based on performance metrics such as F1 score and accuracy. This allows identification of the best-performing classification models.

The results are printed and saved to a CSV file for further analysis and use in the hybrid modelling stage.

```python
# ====================================================
# FINAL RESULTS
# ====================================================

classification_df = pd.DataFrame(classification_results)

classification_df = classification_df.sort_values(by="F1", ascending=False)

print("\n==============================")
print("CLASSIFICATION MODEL RESULTS")
print("==============================")

print(classification_df)

classification_df.to_csv("fursan_model_comparison_classification.csv", index=False)
```

Figure 4.2.4.10.1: Results Table Code

Code Section 11: Final Classification Performance Table

A final evaluation function is implemented to refine classification metrics using macro-averaging. This ensures balanced evaluation across all risk classes. The final classification performance table is generated and exported.

```python
# CLASSIFICATION EVALUATION FUNCTION
# ====================================================
def evaluate_classifier(name, y_true, y_pred, runtime):

    acc = accuracy_score(y_true, y_pred)

    precision = precision_score(
        y_true,
        y_pred,
        average="macro",
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred,
        average="macro",
        zero_division=0
    )

    f1_macro = f1_score(
        y_true,
        y_pred,
        average="macro",
        zero_division=0
    )

    classification_results.append({
        "Model": name,
        "Accuracy": acc,
        "F1_macro": f1_macro,
        "Precision": precision,
        "Recall": recall,
        "Time (sec)": runtime
    })
```

```python
    print(f"{name} completed in {runtime:.2f} seconds")

# ====================================================
# CREATE CLASSIFICATION RESULTS TABLE
# ====================================================
classification_df = pd.DataFrame(classification_results)

classification_df = classification_df[[
    "Model",
    "Accuracy",
    "F1_macro",
    "Precision",
    "Recall",
    "Time (sec)"
]]

classification_df = classification_df.round(6)

# Sort by Accuracy
classification_df = classification_df.sort_values(
    by="Accuracy",
    ascending=False
)

print("\n==============================")
print("CLASSIFICATION MODEL PERFORMANCE")
print("==============================")

print(classification_df)

classification_df.to_csv(
    "table_classification_results.csv",
    index=False
)
```

Figure 4.2.4.10.1: Final Classification Table Code